

Smart Campus Placement Management System

Database Design and Implementation

Database Systems (BACSE202)

Fall Semester 2026-27

Assessment 8 - Project Phase 1

Database: smart_campus_placement

MySQL 8.0.46 Community Edition - InnoDB - utf8mb4

Submitted by: Prayag Krishna

25BCE2223

1. Problem Identification

1.1 Problem Statement

Most college placement cells still run on spreadsheets, shared folders and email. Student profiles, company requirements, eligibility rules, interview schedules and the final placement figures are maintained in separate files, often by different people. This produces a set of recurring and concrete failures.

A student's CGPA corrected in one sheet remains stale in another. Eligibility for every drive has to be re-checked by hand against cut-offs that are written in a notice rather than stored anywhere. Nothing prevents the same student from being recorded twice against the same drive, or a student from being scheduled for a company they never applied to. Skills are kept as free text, so a recruiter asking for candidates who know SQL cannot be answered by a query. Most importantly, an offer that a student accepted and a placement the student actually joined are stored as the same fact, so the end-of-year placement percentage has to be reconciled manually.

The Smart Campus Placement Management System addresses this by moving the placement cell's records into a single normalised relational database. The database becomes the authoritative record of students, recruiting companies, job drives and their eligibility criteria, skills held and skills required, applications, interview rounds, offers and verified placements. Rules that were previously conventions enforced by a person become constraints enforced by the database management system.

1.2 Objectives

1. Model the placement process as a set of relations in Third Normal Form, so that no fact is stored twice and update, insertion and deletion anomalies are structurally avoided.
2. Represent each many-to-many relationship through an explicit associative or bridge relation rather than through repeated or delimited values.
3. Separate an accepted offer from a verified placement, so that placement statistics are reported from confirmed joinings rather than from intentions.
4. Enforce as many business rules as possible inside the schema itself, using primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT, ENUM domains and CHECK constraints.
5. Store no derived value that can be computed from existing data, so that no two columns can disagree.
6. Guarantee referential integrity through InnoDB foreign keys with delete rules chosen per relationship, so that placement history cannot be lost silently.
7. Produce a schema that supports the later phases of the project without structural change: data population and application logic in Phase 2, and testing and demonstration in Phase 3.

2. Requirement Analysis

2.1 Functional Requirements

ID	Requirement
FR1	Maintain student records including registration number, contact details, department, graduating batch, CGPA, active backlogs and school percentages.
FR2	Maintain a department master, since drives are opened to selected branches only.
FR3	Maintain company records together with a single HR contact per company.
FR4	Record job drives conducted by a company, including role, job type, CTC, number of openings, application deadline, drive date and drive status.
FR5	Record the eligibility criteria of a drive: minimum CGPA, maximum backlogs, minimum school percentages, the eligible batch year and whether already-placed students may apply.
FR6	Record which departments a drive is open to.
FR7	Maintain a skill master, record the skills a student holds with a proficiency level, and record the skills a drive requires with a minimum proficiency and a mandatory or preferred flag.
FR8	Record a student's application to a drive, and prevent the same student from applying to the same drive more than once.
FR9	Record the interview rounds of an application, each with a round number, round type, mode, schedule, venue, interviewer, score and result.
FR10	Record at most one offer per application, with the agreed role, CTC, location, offer status and expected joining date.

FR11	Record a verified placement against an accepted offer, with the actual joining date, the employee code issued by the company, the academic year and the verifying officer.
FR12	Support reporting on placement outcomes per batch, department and company without storing any derived totals.

2.2 Non-Functional Requirements

ID	Requirement
NFR1	Integrity. Every relationship is enforced by a named foreign key under the InnoDB engine; MyISAM is not acceptable because it parses foreign keys and ignores them.
NFR2	Correctness by design. The schema is in Third Normal Form, so a fact is stored in exactly one place.
NFR3	Data quality. Value ranges and domains are enforced at the database layer through CHECK constraints and ENUM types rather than being left to the application.
NFR4	Internationalisation. All tables use the utf8mb4 character set with the utf8mb4_0900_ai_ci collation, so names and addresses in any script are stored correctly.
NFR5	Maintainability. Every constraint carries an explicit, descriptive name, so that an error message identifies the rule that was violated.
NFR6	Extensibility. The design supports Phase 2 and Phase 3 without structural change; only data, triggers, views and application code are added.
NFR7	Portability. The scripts are plain MySQL DDL, run locally, and depend on no external service.

3. Entity and Attribute Identification

Thirteen entities were identified. Each one corresponds to exactly one table in the implemented schema. The attribute lists below are taken from database/02_create_tables.sql and were confirmed against the live database.

Table 3.1 - Entities, purpose and keys

Entity	Purpose	Primary Key	Foreign Keys
department	Branch master. Required because a drive is opened to selected branches.	dept_id	-
skill	Skill master shared by students and drives.	skill_id	-
company	Recruiting organisation and its single HR contact.	company_id	-
student	The candidate, with academic and contact details.	student_id	dept_id
job_drive	One job opening conducted by one company.	drive_id	company_id
eligibility_criteria	Optional cut-offs for one drive. The primary key is also the foreign key, which enforces the 1:1.	drive_id	drive_id
drive_eligible_department	Bridge resolving job_drive to department.	drive_id, dept_id	drive_id, dept_id
student_skill	Bridge resolving student to skill, carrying proficiency.	student_id, skill_id	student_id, skill_id
drive_required_skill	Bridge resolving job_drive to skill, carrying the requirement level.	drive_id, skill_id	drive_id, skill_id
application	Associative entity resolving student to job_drive. Has its own children.	application_id	student_id, drive_id
interview	Dependent entity: one round of one application.	interview_id	application_id
offer	Dependent entity: at most one offer per application.	offer_id	application_id
placement_record	Dependent entity: a verified joining against an accepted offer.	placement_id	offer_id

Table 3.2 - Attributes as implemented

Entity	Attributes
department	dept_id, dept_code, dept_name
skill	skill_id, skill_name, skill_category
company	company_id, company_name, industry, website, hr_name, hr_email, hr_phone, city, created_at
student	student_id, reg_no, first_name, last_name, email, phone, dob, gender, dept_id, batch_year, cgpa, active_backlogs, tenth_pct, twelfth_pct, resume_link, created_at
job_drive	drive_id, company_id, job_title, job_type, job_description, ctc_lpa, job_location, num_openings, apply_deadline, drive_date, drive_status, created_at
eligibility_criteria	drive_id, min_cgpa, max_backlogs, min_tenth_pct, min_twelfth_pct, eligible_batch_year, allow_placed_students
drive_eligible_department	drive_id, dept_id
student_skill	student_id, skill_id, proficiency_level
drive_required_skill	drive_id, skill_id, is_mandatory, min_proficiency
application	application_id, student_id, drive_id, applied_on, application_status, remarks
interview	interview_id, application_id, round_no, round_type, mode, scheduled_at, venue, interviewer_name, score, max_score, result, feedback
offer	offer_id, application_id, offer_date, role_title, offered_ctc_lpa, job_location,

	expected_joining_date, offer_status
placement_record	placement_id, offer_id, actual_joining_date, company_employee_code, academic_year, verified_by, verified_on, remarks

Several attributes were deliberately not created because they are derivable from existing data. A student's placement status follows from the existence of a placement_record; a student's age follows from dob; the number of applicants to a drive is a COUNT over application; and the current round of an application is the maximum round_no in interview. Storing any of these would create a second copy of a fact that could disagree with the first. They are to be exposed as views in Phase 2.

4. EER Diagram

The diagram below was produced in MySQL Workbench by reverse-engineering the live smart_campus_placement schema, so it reflects the database as actually created rather than a drawing made beforehand. Solid lines indicate identifying relationships, where the foreign key is part of the child's primary key; dashed lines indicate non-identifying relationships.

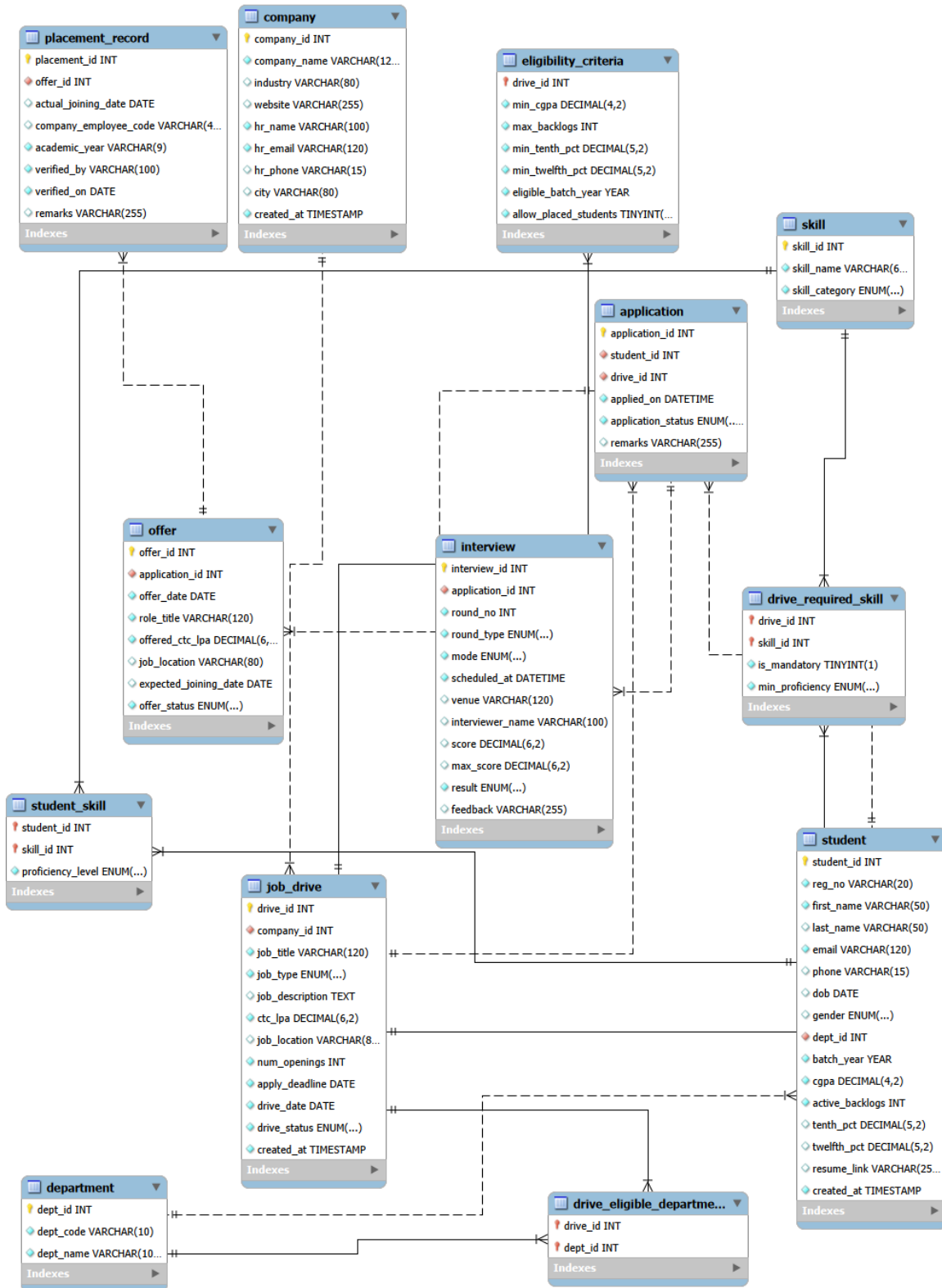


Figure 1: EER diagram of the smart_campus_placement schema, reverse-engineered in MySQL Workbench.

The four many-to-many relationships are visible as the four bridge tables. student_skill, drive_required_skill and drive_eligible_department appear with both foreign keys inside the primary key, which is why Workbench draws them with identifying relationships. application also resolves a many-to-many relationship, between student and job_drive, but it carries its own surrogate key because interview and offer must refer to it through a single column.

5. Relational Model

Primary key attributes are underlined in the convention used below by the marker PK. Foreign keys are marked FK with their referenced relation. Alternate keys enforced by a UNIQUE constraint are marked UK.

```
department(dept_id PK, dept_code UK, dept_name UK)

skill(skill_id PK, skill_name UK, skill_category)

company(company_id PK, company_name UK, industry, website,
        hr_name, hr_email UK, hr_phone, city, created_at)

student(student_id PK, reg_no UK, first_name, last_name, email UK,
        phone UK, dob, gender, dept_id FK -> department, batch_year,
        cgpa, active_backlogs, tenth_pct, twelfth_pct, resume_link,
        created_at)

job_drive(drive_id PK, company_id FK -> company, job_title, job_type,
          job_description, ctc_lpa, job_location, num_openings,
          apply_deadline, drive_date, drive_status, created_at)
          UK (company_id, job_title, drive_date)

eligibility_criteria(drive_id PK FK -> job_drive, min_cgpa,
                    max_backlogs, min_tenth_pct, min_twelfth_pct,
                    eligible_batch_year, allow_placed_students)

drive_eligible_department(drive_id PK FK -> job_drive,
                          dept_id PK FK -> department)

student_skill(student_id PK FK -> student, skill_id PK FK -> skill,
              proficiency_level)

drive_required_skill(drive_id PK FK -> job_drive,
                    skill_id PK FK -> skill,
                    is_mandatory, min_proficiency)

application(application_id PK, student_id FK -> student,
            drive_id FK -> job_drive, applied_on,
            application_status, remarks)
            UK (student_id, drive_id)

interview(interview_id PK, application_id FK -> application, round_no,
          round_type, mode, scheduled_at, venue, interviewer_name,
          score, max_score, result, feedback)
          UK (application_id, round_no)

offer(offer_id PK, application_id FK UK -> application, offer_date,
      role_title, offered_ctc_lpa, job_location,
      expected_joining_date, offer_status)

placement_record(placement_id PK, offer_id FK UK -> offer,
                 actual_joining_date, company_employee_code,
                 academic_year, verified_by, verified_on, remarks)
```

The UNIQUE constraints on offer.application_id and placement_record.offer_id are what reduce those two one-to-many foreign keys to one-to-one relationships. Likewise the primary key of eligibility_criteria being its own foreign key is what makes that relationship one-to-one, with optional participation on the job_drive side because the row need not exist.

6. Normalization

The schema was designed in normalised form rather than decomposed from a single flat table, so this section verifies each relation against the three normal forms using the keys and dependencies that are actually present in database/02_create_tables.sql. Where a design choice avoided a violation, the violation that would otherwise have occurred is stated explicitly.

6.1 First Normal Form

A relation is in 1NF when every attribute holds a single atomic value and no group of attributes repeats within a row. The placement domain contains four natural repeating groups, and each one is held in its own relation rather than in a delimited column.

Repeating group that 1NF forbids	Relation that holds it instead
The skills a student holds	student_skill, one row per (student_id, skill_id)
The skills a drive requires	drive_required_skill, one row per (drive_id, skill_id)
The branches a drive is open to	drive_eligible_department, one row per (drive_id, dept_id)
The interview rounds of an application	interview, one row per (application_id, round_no)

Had the branches been stored as a single column containing a value such as 'CSE, ECE, IT', the relation would not have been in 1NF and the question 'which drives are open to ECE' could not be answered by a join. Evidence of atomicity is visible in the DESCRIBE output in Section 8, where every column of student carries a scalar type and no column is a list.

One case deserves comment. The attribute reg_no often encodes the batch and the branch within the string. It is treated here as an opaque identifier: the schema never decomposes it, and the batch and the branch are stored independently as batch_year and dept_id. Atomicity in 1NF requires that the schema not decompose a value, not that the value be meaningless to a reader.

6.2 Second Normal Form

A relation is in 2NF when it is in 1NF and no non-prime attribute is functionally dependent on only part of a candidate key. 2NF is defined with respect to every candidate key, not only the primary key, so the relations fall into three groups.

Group one consists of the seven relations whose only candidate keys are single columns: department, skill, company, eligibility_criteria, offer and placement_record, together with student. A single-column key has no proper subset, so a partial dependency cannot be formed and 2NF holds by definition.

Group two consists of the three bridge relations whose primary key is composite. Here the condition must be checked rather than assumed.

Relation	Non-prime attributes	Subset test	Result
student_skill	proficiency_level	student_id alone does not determine it, because one student holds different levels in different skills; skill_id alone does not, because different students hold different levels in the same skill.	Full dependency
drive_required_skill	is_mandatory, min_proficiency	drive_id alone does not determine either, because one drive may demand advanced Java and beginner Git; skill_id alone does not, because different drives set different bars for the same skill.	Full dependency
drive_eligible_department	none	The relation is all-key, so there is no non-prime attribute that could depend partially on anything.	Vacuously satisfied

Group three consists of the three relations that carry a composite alternate key created by a UNIQUE constraint. Because 2NF is defined over all candidate keys, these also require a subset test even though their primary key is a single column.

Relation	Composite candidate key	Subset test	Result
----------	-------------------------	-------------	--------

job_drive	(company_id, job_title, drive_date)	No proper subset determines ctc_lpa, num_openings or drive_status: one company runs several drives at different packages, the same title pays differently at different companies, and many drives share a date.	2NF holds
application	(student_id, drive_id)	student_id alone does not determine applied_on or application_status, because one student applies to many drives; drive_id alone does not, because each applicant has an individual date and status.	2NF holds
interview	(application_id, round_no)	application_id alone does not determine round_type, since one application has several round types. round_no alone does not determine round_type either, because round numbering is local to a drive: round 1 is an aptitude test at one company and a technical interview at another.	2NF holds

6.3 Third Normal Form

A relation is in 3NF when it is in 2NF and no non-prime attribute is transitively dependent on a candidate key, that is, when no non-key attribute determines another non-key attribute. Two transitive dependencies would arise naturally in this domain, and the schema removes both by placing the determinant in its own relation.

The first concerns the student's branch. A flat design would store dept_code and dept_name alongside the student, giving student_id → dept_code → dept_name. The branch name would then be repeated once per student and a correction would have to be applied to every row. The schema stores only dept_id as a foreign key, so dept_name is not an attribute of student at all and the chain cannot exist.

The second concerns company details. Storing company_name or hr_email on job_drive or on application would give drive_id → company_id → company_name. Both relations store only the key that leads to the company, so the company's attributes appear exactly once, in company.

The company relation itself repays a closer look, because it contains the dependency hr_email → hr_name, which appears at first sight to be a transitive dependency between two non-key attributes. It is not. The column hr_email is declared NOT NULL and UNIQUE, which makes it a candidate key of company. The relation therefore has three candidate keys, {company_id}, {company_name} and {hr_email}, and the dependency is from a candidate key rather than from a non-prime attribute. 3NF requires that for every non-trivial dependency X → A, either X is a superkey or A is prime; the first branch is satisfied. The UNIQUE constraint is therefore not decoration. If it were removed, hr_email would become a non-prime attribute, the dependency would become transitive, and company would have to be decomposed to reach 3NF. The design assumes that one HR contact represents exactly one company.

A related point applies to student. The column phone carries a UNIQUE constraint but is nullable, and a candidate key must be NOT NULL. phone is therefore a uniqueness constraint and not a candidate key, while reg_no and email, both NOT NULL and UNIQUE, are genuine alternate keys.

Finally, the separation of offer from placement_record is a 3NF-preserving decision rather than a duplication. An accepted offer and a verified placement are different facts recorded by different parties at different times: an offer status of 'Accepted' records the student's intention, whereas a placement_record is the placement cell's confirmation that the student joined. Every placement has an offer behind it, but not every accepted offer becomes a placement. placement_record therefore stores no attribute that already exists in offer; it holds only the joining date, the employee code, the academic year and the verification details, and reaches the company and the student by joining through offer and application.

Table 6.1 - Normal form verdict for each relation

Relation	Primary key	Basis for 2NF	3NF
department	dept_id	single-column key	Satisfied
skill	skill_id	single-column key	Satisfied
company	company_id	single-column key	Satisfied
student	student_id	single-column key	Satisfied
job_drive	drive_id	subset test on composite UK	Satisfied
eligibility_criteria	drive_id	single-column key	Satisfied

drive_eligible_department	drive_id, dept_id	all-key, vacuous	Satisfied
student_skill	student_id, skill_id	subset test, full dependency	Satisfied
drive_required_skill	drive_id, skill_id	subset test, full dependency	Satisfied
application	application_id	subset test on composite UK	Satisfied
interview	interview_id	subset test on composite UK	Satisfied
offer	offer_id	single-column key	Satisfied
placement_record	placement_id	single-column key	Satisfied

All thirteen relations are in Third Normal Form. Each one also satisfies Boyce-Codd Normal Form, since in every relation each determinant is a candidate key. One further point is worth recording: the eligible departments of a drive and the required skills of a drive are two independent multivalued facts about the same drive. Combining them into a single relation would produce a multivalued dependency and a violation of Fourth Normal Form, and would generate one row for every combination of branch and skill. Keeping drive_eligible_department and drive_required_skill as two separate binary relations avoids this.

7. Database Design and DDL

7.1 Database management system

The database is implemented on MySQL 8.0.46 Community Edition, running locally. Every table uses the InnoDB storage engine, declared explicitly in each CREATE TABLE statement, because InnoDB is the engine that enforces foreign keys; MyISAM accepts foreign key syntax and silently ignores it. The schema and all tables use the utf8mb4 character set with the utf8mb4_0900_ai_ci collation. The MySQL version matters for one further reason: CHECK constraints are only enforced from version 8.0.16 onward, and are parsed and ignored by earlier releases.

7.2 Script organisation

File	Contents
01_create_database.sql	Creates the schema with its character set and collation, then reports the server version, engine and collation actually in effect.
02_create_tables.sql	Creates the thirteen tables with every constraint a table can enforce on its own: PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT, ENUM domains and CHECK.
03_constraints.sql	Applies referential integrity as fourteen named FOREIGN KEY constraints by ALTER TABLE.
04_verify.sql	Verification queries against INFORMATION_SCHEMA used to produce the evidence in Section 8.

Foreign keys are applied in a separate file so that table creation order never blocks a rerun and so that every referential constraint carries an explicit name. Script 02 drops the thirteen tables in reverse dependency order before creating them, which makes the pair of scripts repeatable.

7.3 Primary key strategy

Ten relations use a single-column surrogate key with AUTO_INCREMENT. Three bridge relations use the natural composite key formed by their two foreign keys: drive_eligible_department, student_skill and drive_required_skill. eligibility_criteria is a special case in which the primary key, drive_id, is simultaneously the foreign key to job_drive; this is what enforces the one-to-one relationship in the schema rather than in application code.

application resolves a many-to-many relationship and could have used the composite key (student_id, drive_id). A surrogate application_id was chosen instead so that interview, offer and ultimately placement_record refer to it through a single column rather than propagating two columns down the chain. The natural key is retained as a UNIQUE constraint, so no information is lost.

7.4 Referential integrity

Fourteen foreign keys connect the thirteen relations. The delete rule was chosen per relationship rather than applied uniformly. CASCADE is used where a child row has no meaning without its parent; RESTRICT is used where the row is placement history that must not be lost silently. ON UPDATE is RESTRICT throughout, because every referenced key is a surrogate that never changes.

Delete rule	Foreign keys	Rationale
CASCADE	fk_criteria_drive, fk_dedept_drive, fk_driveskill_drive, fk_studentskill_student, fk_interview_application	The child is a detail of its parent: cut-offs, bridge rows and interview rounds cease to exist when the drive, student or application does.
RESTRICT	fk_student_department, fk_drive_company, fk_dedept_department, fk_driveskill_skill, fk_studentskill_skill, fk_application_student, fk_application_drive, fk_offer_application, fk_placement_offer	Master data and placement history. A department, company, student, application or offer that is referenced cannot be deleted.

The foreign key graph is acyclic. Its topological order is department, skill and company; then student and job_drive; then eligibility_criteria, the three bridges and application; then interview and offer; and finally placement_record. This is also the table creation order, and its reverse is the drop order.

7.5 UNIQUE constraints

Thirteen UNIQUE constraints are declared. Ten are single-column alternate keys or uniqueness rules; three are composite and encode business rules directly.

Constraint	Columns	Rule enforced
uq_application_student_drive	student_id, drive_id	A student cannot apply to the same drive twice.
uq_interview_app_round	application_id, round_no	An application cannot have two rounds with the same number.
uq_job_drive_company_title_date	company_id, job_title, drive_date	The same drive cannot be entered twice.
uq_offer_application	application_id	An application yields at most one offer, which makes the relationship 1:1.
uq_placement_offer	offer_id	An offer yields at most one placement record.

7.6 CHECK constraints and domains

Twenty-one CHECK constraints are declared, each with an explicit name. MySQL requires CHECK constraint names to be unique across the whole database rather than per table, so every name is prefixed with its table. A CHECK in MySQL may reference only columns of the row being written, which defines the limit of what this layer can do.

Category	Examples
Numeric range	chk_student_cgpa and chk_ec_min_cgpa restrict CGPA to 0 to 10; chk_student_tenth and chk_student_twelfth restrict percentages to 0 to 100; chk_student_backlogs requires a non-negative count.
Positive value	chk_job_drive_ctc and chk_offer_ctc require a positive package; chk_job_drive_openings requires at least one opening; chk_interview_round_no requires a positive round number.
Date ordering	chk_job_drive_dates requires apply_deadline on or before drive_date; chk_offer_dates requires the expected joining date to fall on or after the offer date.
Cross-column	chk_interview_score_max requires score to be at most max_score; chk_interview_venue requires an offline round to name a venue.
Format	chk_placement_academic_year requires academic_year to match the pattern YYYY-YYYY.

Eleven columns additionally restrict their domain through an ENUM type, covering gender, job type, drive status, skill category, proficiency levels, application status, round type, interview mode, interview result and offer status. DEFAULT values are supplied for timestamps, status columns, CGPA, backlog count, number of openings and the two boolean flags.

7.7 Rules deferred to later phases

Several genuine business rules cannot be expressed as a CHECK, because each one must read another table or another row. These are documented rather than implemented, and belong to Phase 2 as triggers or application logic: that a placement_record may exist only against an accepted offer; that an application must fall on or before the drive's deadline; that an applicant must satisfy the drive's eligibility_criteria and belong to an eligible department; and that drive_status must not move backwards. Phase 1 contains no triggers and no sample data.

7.8 Representative DDL

The following extracts are taken verbatim from the project scripts. Appendix A contains the complete files.

```
CREATE TABLE student (
  student_id      INT          NOT NULL AUTO_INCREMENT,
  reg_no          VARCHAR(20)  NOT NULL,
  first_name      VARCHAR(50)  NOT NULL,
  email           VARCHAR(120) NOT NULL,
  dept_id         INT          NOT NULL,
  batch_year      YEAR         NOT NULL,
  cgpa            DECIMAL(4,2) NOT NULL DEFAULT 0.00,
  active_backlogs INT          NOT NULL DEFAULT 0,
  created_at      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT pk_student      PRIMARY KEY (student_id),
  CONSTRAINT uq_student_reg_no UNIQUE      (reg_no),
  CONSTRAINT uq_student_email UNIQUE      (email),
  CONSTRAINT chk_student_cgpa CHECK (cgpa BETWEEN 0.00 AND 10.00),
```

```

        CONSTRAINT chk_student_backlogs CHECK (active_backlogs >= 0)
    ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

-- A bridge relation resolving a many-to-many relationship
CREATE TABLE student_skill (
    student_id      INT NOT NULL,
    skill_id        INT NOT NULL,
    proficiency_level ENUM('Beginner', 'Intermediate', 'Advanced')
                    NOT NULL DEFAULT 'Beginner',

    CONSTRAINT pk_student_skill PRIMARY KEY (student_id, skill_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

-- Referential integrity, applied in 03_constraints.sql
ALTER TABLE application
    ADD CONSTRAINT fk_application_student
        FOREIGN KEY (student_id) REFERENCES student (student_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT;

```

8. Implementation Evidence

The screenshots below were captured from MySQL Workbench connected to the live smart_campus_placement database after the three scripts had been executed.

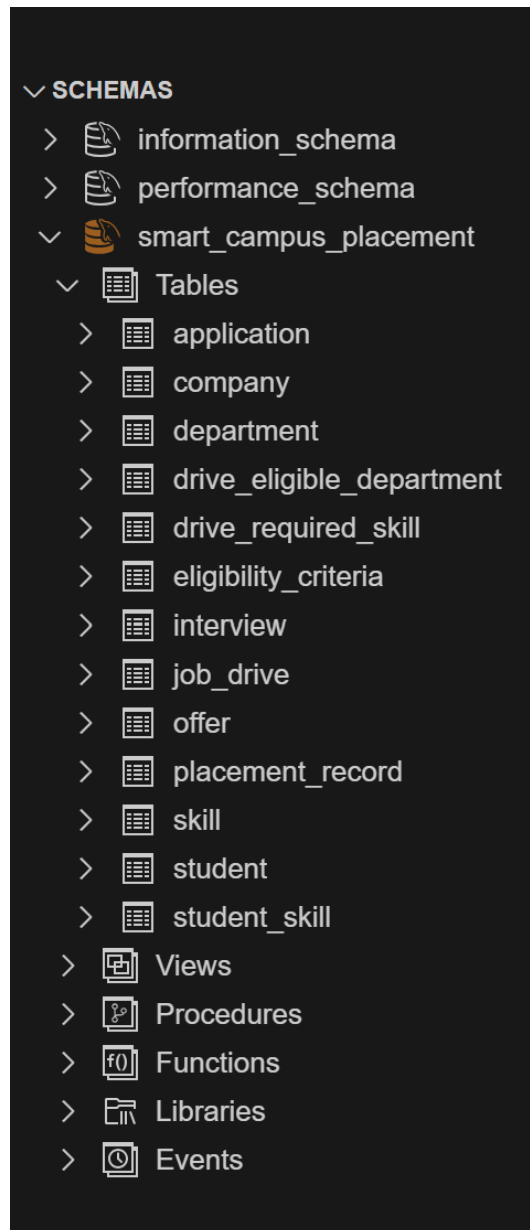


Figure 2: MySQL Workbench schema showing the project tables.

Untitled-1.sql ●

1

• USE smart_campus_placement;

2

• SHOW TABLES;

Output

Result #2

Tables_in_smart_campus_placement	
application	
company	
department	
drive_eligible_department	
drive_required_skill	
eligibility_criteria	
interview	
job_drive	
offer	
placement_record	
skill	
student	
student_skill	

Figure 3: SHOW TABLES output.

Untitled-1.sql •

1 • DESCRIBE student;

Output

Result #1

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	auto_increment
reg_no	varchar(20)	NO	UNI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	YES		NULL	
email	varchar(120)	NO	UNI	NULL	
phone	varchar(15)	YES	UNI	NULL	
dob	date	YES		NULL	
gender	enum('Male','Female','Other')	YES		NULL	
dept_id	int	NO	MUL	NULL	
batch_year	year	NO		NULL	
cgpa	decimal(4,2)	NO		0.00	
active_backlogs	int	NO		0	
tenth_pct	decimal(5,2)	YES		NULL	
twelfth_pct	decimal(5,2)	YES		NULL	
resume_link	varchar(255)	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Figure 4: student table structure.

Untitled-1.sql •

1 • DESCRIBE student_skill;

Output

Result #1

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	
skill_id	int	NO	PRI	NULL	
proficiency_level	enum('Beginner','Intermediate','Advanced')	NO		Beginner	

Figure 5: student_skill composite primary key.

Untitled-1.sql				
1	SELECT			
2	TABLE_NAME,			
3	COLUMN_NAME,			
4	CONSTRAINT_NAME,			
5	REFERENCED_TABLE_NAME,			
6	REFERENCED_COLUMN_NAME			
7	FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE			
8	WHERE TABLE_SCHEMA = 'smart_campus_placement'			
9	AND REFERENCED_TABLE_NAME IS NOT NULL			
10	ORDER BY TABLE_NAME, COLUMN_NAME;			

Output	Result #1			
TABLE_NAME	COLUMN_NAME	CONSTRAINT_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
application	drive_id	fk_application_drive	job_drive	drive_id
application	student_id	fk_application_student	student	student_id
drive_eligible_department	dept_id	fk_ddept_department	department	dept_id
drive_eligible_department	drive_id	fk_ddept_drive	job_drive	drive_id
drive_required_skill	drive_id	fk_driveskill_drive	job_drive	drive_id
drive_required_skill	skill_id	fk_driveskill_skill	skill	skill_id
eligibility_criteria	drive_id	fk_criteria_drive	job_drive	drive_id
interview	application_id	fk_interview_application	application	application_id
job_drive	company_id	fk_drive_company	company	company_id
offer	application_id	fk_offer_application	application	application_id
placement_record	offer_id	fk_placement_offer	offer	offer_id
student	dept_id	fk_student_department	department	dept_id
student_skill	skill_id	fk_studentskill_skill	skill	skill_id
student_skill	student_id	fk_studentskill_student	student	student_id

Figure 6: foreign-key metadata from INFORMATION_SCHEMA.

Figures 2 and 3 confirm that all thirteen tables exist in the schema. Figure 4 shows the structure of student, in which student_id is the primary key, reg_no and email carry UNIQUE constraints, dept_id is indexed as a foreign key, and every column holds an atomic value. Figure 5 shows PRI against both student_id and skill_id, which is the composite primary key that resolves the student-to-skill many-to-many relationship. Figure 6 lists the fourteen foreign keys with their referenced tables and columns.

9. Conclusion

Phase 1 has produced a complete and verified database design for the Smart Campus Placement Management System. The placement process was analysed into thirteen entities; the four many-to-many relationships were resolved by explicit bridge and associative relations; and the resulting schema was checked relation by relation against the first, second and third normal forms using its actual candidate keys and functional dependencies, with each relation also satisfying Boyce-Codd Normal Form.

The design was implemented in MySQL 8.0.46 on the InnoDB engine with the utf8mb4 character set. Thirteen tables were created, carrying thirteen primary keys, fourteen named foreign keys, thirteen UNIQUE constraints, twenty-one named CHECK constraints and eleven ENUM domains. The scripts execute without error, and the table structures, the composite primary key of a bridge relation and the complete foreign key metadata have been captured from the running database as evidence.

The schema contains no triggers and no sample data, both of which belong to Phase 2, and no derived attributes, which will be exposed as views. The cross-table rules that MySQL cannot express as a CHECK constraint have been identified and documented so that Phase 2 can implement them as triggers or application logic. The design is therefore

ready for data population, application development and connectivity in Phase 2, and for testing and demonstration in Phase 3, without structural change.

Appendix A - SQL DDL Scripts

The complete contents of the three DDL scripts, reproduced verbatim from the project. No credentials appear in any of them.

A.1 database/01_create_database.sql

```
-- =====
-- SMART CAMPUS PLACEMENT MANAGEMENT SYSTEM
-- Course : BACSE202 - Database Systems
-- Phase  : 1 (Database Design and Table Creation)
--
-- File   : 01_create_database.sql
-- Purpose: Create the project schema with the required character set.
-- Target : MySQL 8.0.16 or later (CHECK constraints are enforced from 8.0.16)
-- Engine : InnoDB (set per table in 02_create_tables.sql)
--
-- Run order: 01 -> 02 -> 03 -> 04
-- =====

CREATE DATABASE IF NOT EXISTS smart_campus_placement
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_0900_ai_ci;

USE smart_campus_placement;

-- Confirm what the server actually is, so the CHECK support claim is verifiable.
SELECT
    VERSION()           AS mysql_version,
    DATABASE()          AS current_database,
    @@default_storage_engine AS default_engine,
    @@character_set_database AS db_charset,
    @@collation_database   AS db_collation;
```

A.2 database/02_create_tables.sql

```
-- =====
-- SMART CAMPUS PLACEMENT MANAGEMENT SYSTEM
-- File   : 02_create_tables.sql
--
-- Purpose: Create all 13 base tables.
--         This file carries every constraint that belongs to a table on its
--         own: PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT, ENUM domains and
--         CHECK constraints.
--         REFERENTIAL integrity (FOREIGN KEY) is applied separately in
--         03_constraints.sql, so table creation order never blocks a rerun.
--
-- Engine : InnoDB (MyISAM silently ignores FOREIGN KEY - do not change this)
-- Charset: utf8mb4 with collation utf8mb4_0900_ai_ci
-- Target : MySQL 8.0.16+
--
-- Note on constraint names: in MySQL a CHECK constraint name must be unique
-- across the whole database, not just the table. Every CHECK below is
-- therefore prefixed with its table name.
-- =====

USE smart_campus_placement;

-- -----
-- Drop in reverse dependency order so this script can be rerun safely.
-- -----

DROP TABLE IF EXISTS placement_record;
DROP TABLE IF EXISTS offer;
DROP TABLE IF EXISTS interview;
DROP TABLE IF EXISTS application;
DROP TABLE IF EXISTS drive_required_skill;
DROP TABLE IF EXISTS student_skill;
DROP TABLE IF EXISTS drive_eligible_department;
DROP TABLE IF EXISTS eligibility_criteria;
DROP TABLE IF EXISTS job_drive;
DROP TABLE IF EXISTS student;
DROP TABLE IF EXISTS company;
DROP TABLE IF EXISTS skill;
DROP TABLE IF EXISTS department;
```

```

-- =====
-- LEVEL 0 : independent master tables
-- =====

-- -----
-- 1. department (strong entity)
-- Branch master. Needed because a drive is open to selected branches only.
-- Candidate keys: {dept_id}, {dept_code}, {dept_name}
-- -----

CREATE TABLE department (
    dept_id INT NOT NULL AUTO_INCREMENT,
    dept_code VARCHAR(10) NOT NULL,
    dept_name VARCHAR(100) NOT NULL,

    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_department_code UNIQUE (dept_code),
    CONSTRAINT uq_department_name UNIQUE (dept_name)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

-- -----

-- 2. skill (strong entity)
-- Skill master. Referenced by student_skill and drive_required_skill.
-- Candidate keys: {skill_id}, {skill_name}
-- -----

CREATE TABLE skill (
    skill_id INT NOT NULL AUTO_INCREMENT,
    skill_name VARCHAR(60) NOT NULL,
    skill_category ENUM('Programming', 'Database', 'Web', 'Tools', 'Soft Skill', 'Other')
    NOT NULL DEFAULT 'Other',

    CONSTRAINT pk_skill PRIMARY KEY (skill_id),
    CONSTRAINT uq_skill_name UNIQUE (skill_name)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

-- -----

-- 3. company (strong entity)
-- Candidate keys: {company_id}, {company_name}, {hr_email}
-- hr_email is NOT NULL + UNIQUE on purpose. That makes it a candidate key,
-- which is why hr_email -> hr_name is a key dependency and NOT a 3NF
-- violation. Assumption: one HR contact represents exactly one company.
-- -----

CREATE TABLE company (
    company_id INT NOT NULL AUTO_INCREMENT,
    company_name VARCHAR(120) NOT NULL,
    industry VARCHAR(80) NULL,
    website VARCHAR(255) NULL,
    hr_name VARCHAR(100) NOT NULL,
    hr_email VARCHAR(120) NOT NULL,
    hr_phone VARCHAR(15) NULL,
    city VARCHAR(80) NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_company PRIMARY KEY (company_id),
    CONSTRAINT uq_company_name UNIQUE (company_name),
    CONSTRAINT uq_company_hr_email UNIQUE (hr_email)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

-- =====

-- LEVEL 1 : depend on the masters above
-- =====

-- -----

-- 4. student (strong entity)
-- Candidate keys: {student_id}, {reg_no}, {email}
-- phone is UNIQUE but NULLABLE, so it is a uniqueness constraint and NOT a
-- candidate key (a candidate key must be NOT NULL).
-- No is_placed / age column: both are derived and are exposed as views.
-- -----

CREATE TABLE student (
    student_id INT NOT NULL AUTO_INCREMENT,
    reg_no VARCHAR(20) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NULL,
    email VARCHAR(120) NOT NULL,
    phone VARCHAR(15) NULL,

```

```

dob            DATE            NULL,
gender         ENUM('Male','Female','Other') NULL,
dept_id        INT             NOT NULL,
batch_year     YEAR            NOT NULL,
cgpa           DECIMAL(4,2) NOT NULL DEFAULT 0.00,
active_backlogs INT          NOT NULL DEFAULT 0,
tenth_pct      DECIMAL(5,2) NULL,
twelfth_pct    DECIMAL(5,2) NULL,
resume_link    VARCHAR(255) NULL,
created_at     TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT pk_student PRIMARY KEY (student_id),
CONSTRAINT uq_student_reg_no UNIQUE (reg_no),
CONSTRAINT uq_student_email UNIQUE (email),
CONSTRAINT uq_student_phone UNIQUE (phone),

CONSTRAINT chk_student_cgpa CHECK (cgpa BETWEEN 0.00 AND 10.00),
CONSTRAINT chk_student_backlogs CHECK (active_backlogs >= 0),
CONSTRAINT chk_student_tenth CHECK (tenth_pct IS NULL OR tenth_pct BETWEEN 0.00 AND 100.00),
CONSTRAINT chk_student_twelfth CHECK (twelfth_pct IS NULL OR twelfth_pct BETWEEN 0.00 AND 100.00),
CONSTRAINT chk_student_batch_year CHECK (batch_year BETWEEN 2000 AND 2100)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- 5. job_drive (strong entity)
--   Candidate keys: {drive_id} and {company_id, job_title, drive_date}
--   The composite alternate key is why 2NF had to be checked by subset
--   testing here instead of claimed automatically.
-----

```

```

CREATE TABLE job_drive (
  drive_id      INT          NOT NULL AUTO_INCREMENT,
  company_id    INT          NOT NULL,
  job_title     VARCHAR(120) NOT NULL,
  job_type      ENUM('Internship','Full Time','Intern + FTE')
               NOT NULL DEFAULT 'Full Time',
  job_description TEXT       NULL,
  ctc_lpa       DECIMAL(6,2) NOT NULL,
  job_location  VARCHAR(80)  NULL,
  num_openings  INT          NOT NULL DEFAULT 1,
  apply_deadline DATE       NOT NULL,
  drive_date    DATE       NOT NULL,
  drive_status  ENUM('Upcoming','Open','Closed','Completed','Cancelled')
               NOT NULL DEFAULT 'Upcoming',
  created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT pk_job_drive PRIMARY KEY (drive_id),
  CONSTRAINT uq_job_drive_company_title_date
    UNIQUE (company_id, job_title, drive_date),

  CONSTRAINT chk_job_drive_ctc CHECK (ctc_lpa > 0),
  CONSTRAINT chk_job_drive_openings CHECK (num_openings > 0),
  CONSTRAINT chk_job_drive_dates CHECK (apply_deadline <= drive_date)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- LEVEL 2 : dependents and bridges
--
-----

```

```

-----
-- 6. eligibility_criteria (optional 1:1 dependent entity)
--   PRIMARY KEY is also the FOREIGN KEY. That is what enforces 1:1 in the
--   schema itself. The row is optional, so a drive may have no cut-offs.
--   Assumption A1: one drive targets exactly one eligible batch year.
-----

```

```

CREATE TABLE eligibility_criteria (
  drive_id      INT          NOT NULL,
  min_cgpa      DECIMAL(4,2) NOT NULL DEFAULT 0.00,
  max_backlogs  INT          NOT NULL DEFAULT 0,
  min_tenth_pct DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  min_twelfth_pct DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  eligible_batch_year YEAR    NOT NULL,
  allow_placed_students BOOLEAN NOT NULL DEFAULT FALSE,

  CONSTRAINT pk_eligibility_criteria PRIMARY KEY (drive_id),

  CONSTRAINT chk_ec_min_cgpa CHECK (min_cgpa BETWEEN 0.00 AND 10.00),

```

```

        CONSTRAINT chk_ec_max_backlogs CHECK (max_backlogs >= 0),
        CONSTRAINT chk_ec_min_tenth CHECK (min_tenth_pct BETWEEN 0.00 AND 100.00),
        CONSTRAINT chk_ec_min_twelfth CHECK (min_twelfth_pct BETWEEN 0.00 AND 100.00),
        CONSTRAINT chk_ec_batch_year CHECK (eligible_batch_year BETWEEN 2000 AND 2100)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- 7. drive_eligible_department (pure bridge)
-- Resolves the M:N job_drive <-> department.
-- All-key relation: no non-prime attribute exists, so 2NF and 3NF hold
-- vacuously. This table is the 1NF fix for a comma separated branch list.
-----

```

```

CREATE TABLE drive_eligible_department (
    drive_id INT NOT NULL,
    dept_id INT NOT NULL,

    CONSTRAINT pk_drive_eligible_department PRIMARY KEY (drive_id, dept_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- 8. student_skill (associative bridge)
-- Resolves the M:N student <-> skill, and carries proficiency_level.
-- 1NF fix for storing skills as 'Java, SQL, Python' in one column.
-----

```

```

CREATE TABLE student_skill (
    student_id INT NOT NULL,
    skill_id INT NOT NULL,
    proficiency_level ENUM('Beginner', 'Intermediate', 'Advanced')
        NOT NULL DEFAULT 'Beginner',

    CONSTRAINT pk_student_skill PRIMARY KEY (student_id, skill_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- 9. drive_required_skill (associative bridge)
-- Resolves the M:N job_drive <-> skill.
-- Kept separate from drive_eligible_department on purpose: merging the two
-- multivalued facts into one table would create a 4NF violation.
-----

```

```

CREATE TABLE drive_required_skill (
    drive_id INT NOT NULL,
    skill_id INT NOT NULL,
    is_mandatory BOOLEAN NOT NULL DEFAULT TRUE,
    min_proficiency ENUM('Beginner', 'Intermediate', 'Advanced')
        NOT NULL DEFAULT 'Beginner',

    CONSTRAINT pk_drive_required_skill PRIMARY KEY (drive_id, skill_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- 10. application (associative entity)
-- Resolves the M:N student <-> job_drive.
-- Candidate keys: {application_id} and {student_id, drive_id}
-- The UNIQUE on (student_id, drive_id) is the business rule
-- "one student cannot apply twice to the same drive".
-- No ctc column here: that would duplicate job_drive.ctc_lpa.
-----

```

```

CREATE TABLE application (
    application_id INT NOT NULL AUTO_INCREMENT,
    student_id INT NOT NULL,
    drive_id INT NOT NULL,
    applied_on DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    application_status ENUM('Applied', 'Under Review', 'Shortlisted',
        'Rejected', 'Selected', 'Withdrawn')
        NOT NULL DEFAULT 'Applied',
    remarks VARCHAR(255) NULL,

    CONSTRAINT pk_application PRIMARY KEY (application_id),
    CONSTRAINT uq_application_student_drive UNIQUE (student_id, drive_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-----
-- LEVEL 3 and 4 : results of an application

```

```

-- =====
-- -----
-- 11. interview (dependent entity)
-- One round of one application. It points at application_id only, never
-- at student_id + drive_id, so an interview without an application is
-- impossible.
-- Candidate keys: {interview_id} and {application_id, round_no}
-- -----
CREATE TABLE interview (
    interview_id INT NOT NULL AUTO_INCREMENT,
    application_id INT NOT NULL,
    round_no INT NOT NULL,
    round_type ENUM('Aptitude Test','Technical Test','Group Discussion',
        'Technical Interview','HR Interview','Managerial')
        NOT NULL,
    `mode` ENUM('Online','Offline') NOT NULL DEFAULT 'Offline',
    scheduled_at DATETIME NOT NULL,
    venue VARCHAR(120) NULL,
    interviewer_name VARCHAR(100) NULL,
    score DECIMAL(6,2) NULL,
    max_score DECIMAL(6,2) NULL,
    result ENUM('Pending','Pass','Fail','Absent')
        NOT NULL DEFAULT 'Pending',
    feedback VARCHAR(255) NULL,

    CONSTRAINT pk_interview PRIMARY KEY (interview_id),
    CONSTRAINT uq_interview_app_round UNIQUE (application_id, round_no),

    CONSTRAINT chk_interview_round_no CHECK (round_no > 0),
    CONSTRAINT chk_interview_score CHECK (score IS NULL OR score >= 0),
    CONSTRAINT chk_interview_max_score CHECK (max_score IS NULL OR max_score > 0),
    CONSTRAINT chk_interview_score_max CHECK (score IS NULL OR max_score IS NULL
        OR score <= max_score),
    -- An offline round must name a venue. An online round need not.
    CONSTRAINT chk_interview_venue CHECK (`mode` = 'Online' OR venue IS NOT NULL)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-- -----
-- 12. offer (1:1 dependent entity)
-- UNIQUE (application_id) enforces at most one offer per application.
-- role_title / job_location / offered_ctc_lpa are a snapshot of the agreed
-- terms, the same reason an order line stores its own unit price. They are
-- not duplicates of job_drive.
-- Candidate keys: {offer_id}, {application_id}
-- -----
CREATE TABLE offer (
    offer_id INT NOT NULL AUTO_INCREMENT,
    application_id INT NOT NULL,
    offer_date DATE NOT NULL,
    role_title VARCHAR(120) NOT NULL,
    offered_ctc_lpa DECIMAL(6,2) NOT NULL,
    job_location VARCHAR(80) NULL,
    expected_joining_date DATE NULL,
    offer_status ENUM('Offered','Accepted','Declined','Expired','Revoked')
        NOT NULL DEFAULT 'Offered',

    CONSTRAINT pk_offer PRIMARY KEY (offer_id),
    CONSTRAINT uq_offer_application UNIQUE (application_id),

    CONSTRAINT chk_offer_ctc CHECK (offered_ctc_lpa > 0),
    CONSTRAINT chk_offer_dates CHECK (expected_joining_date IS NULL
        OR expected_joining_date >= offer_date)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

```

```

-- -----
-- 13. placement_record (1:1 dependent entity)
-- A verified placement, which is a different fact from an accepted offer:
-- an accepted offer is the student's intention, a placement record is the
-- placement cell's verified outcome.
-- Holds ZERO columns that already exist in offer. No CTC, no company,
-- no student. Those are reached by joining through offer.
-- Candidate keys: {placement_id}, {offer_id}
-- -----
CREATE TABLE placement_record (
    placement_id INT NOT NULL AUTO_INCREMENT,

```

```

offer_id            INT            NOT NULL,
actual_joining_date DATE          NULL,
company_employee_code VARCHAR(40)  NULL,
academic_year       VARCHAR(9)    NOT NULL,
verified_by         VARCHAR(100)  NOT NULL,
verified_on         DATE          NOT NULL DEFAULT (CURRENT_DATE),
remarks             VARCHAR(255)  NULL,

CONSTRAINT pk_placement_record PRIMARY KEY (placement_id),
CONSTRAINT uq_placement_offer  UNIQUE      (offer_id),

-- Format 'YYYY-YYYY', for example '2026-2027'.
CONSTRAINT chk_placement_academic_year
CHECK (academic_year REGEXP '^[0-9]{4}-[0-9]{4}$')
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;

SELECT '02_create_tables.sql completed' AS status;

```

A.3 database/03_constraints.sql

```

-- =====
-- SMART CAMPUS PLACEMENT MANAGEMENT SYSTEM
-- File   : 03_constraints.sql
--
-- Purpose: Apply referential integrity. 14 named FOREIGN KEY constraints.
--
-- ON DELETE policy, decided per relationship:
-- CASCADE -> the child row has no meaning without its parent
--           (criteria, bridge rows, interview rounds)
-- RESTRICT -> the row is placement history and must never be lost silently
--           (students, companies, drives, applications, offers, masters)
--
-- ON UPDATE is RESTRICT everywhere, because every parent key is an
-- AUTO_INCREMENT surrogate that never changes.
--
-- Phase 1 scope note: cross-table business rules (eligibility enforcement,
-- "a placement record requires an accepted offer", deadline checks) CANNOT be
-- expressed as a MySQL CHECK, because a CHECK may only read columns of the row
-- being written. Those rules are documented in database/README.md and are
-- deliberately deferred to Phase 2 as triggers / application logic.
-- =====

USE smart_campus_placement;

-- --- student -> department -----
ALTER TABLE student
ADD CONSTRAINT fk_student_department
    FOREIGN KEY (dept_id) REFERENCES department (dept_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- job_drive -> company -----
ALTER TABLE job_drive
ADD CONSTRAINT fk_drive_company
    FOREIGN KEY (company_id) REFERENCES company (company_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- eligibility_criteria -> job_drive (1:1, PK is also the FK) -----
ALTER TABLE eligibility_criteria
ADD CONSTRAINT fk_criteria_drive
    FOREIGN KEY (drive_id) REFERENCES job_drive (drive_id)
    ON DELETE CASCADE ON UPDATE RESTRICT;

-- --- drive_eligible_department -> job_drive, department -----
ALTER TABLE drive_eligible_department
ADD CONSTRAINT fk_ddept_drive
    FOREIGN KEY (drive_id) REFERENCES job_drive (drive_id)
    ON DELETE CASCADE ON UPDATE RESTRICT;

ALTER TABLE drive_eligible_department
ADD CONSTRAINT fk_ddept_department
    FOREIGN KEY (dept_id) REFERENCES department (dept_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

```

```

-- --- student_skill -> student, skill -----
ALTER TABLE student_skill
  ADD CONSTRAINT fk_studentskill_student
    FOREIGN KEY (student_id) REFERENCES student (student_id)
    ON DELETE CASCADE ON UPDATE RESTRICT;

ALTER TABLE student_skill
  ADD CONSTRAINT fk_studentskill_skill
    FOREIGN KEY (skill_id) REFERENCES skill (skill_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- drive_required_skill -> job_drive, skill -----
ALTER TABLE drive_required_skill
  ADD CONSTRAINT fk_driveskill_drive
    FOREIGN KEY (drive_id) REFERENCES job_drive (drive_id)
    ON DELETE CASCADE ON UPDATE RESTRICT;

ALTER TABLE drive_required_skill
  ADD CONSTRAINT fk_driveskill_skill
    FOREIGN KEY (skill_id) REFERENCES skill (skill_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- application -> student, job_drive -----
ALTER TABLE application
  ADD CONSTRAINT fk_application_student
    FOREIGN KEY (student_id) REFERENCES student (student_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

ALTER TABLE application
  ADD CONSTRAINT fk_application_drive
    FOREIGN KEY (drive_id) REFERENCES job_drive (drive_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- interview -> application -----
ALTER TABLE interview
  ADD CONSTRAINT fk_interview_application
    FOREIGN KEY (application_id) REFERENCES application (application_id)
    ON DELETE CASCADE ON UPDATE RESTRICT;

-- --- offer -> application (1:1 via UNIQUE on application_id) -----
ALTER TABLE offer
  ADD CONSTRAINT fk_offer_application
    FOREIGN KEY (application_id) REFERENCES application (application_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

-- --- placement_record -> offer (1:1 via UNIQUE on offer_id) -----
ALTER TABLE placement_record
  ADD CONSTRAINT fk_placement_offer
    FOREIGN KEY (offer_id) REFERENCES offer (offer_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT;

SELECT '03_constraints.sql completed' AS status;

```